

AI -Assistant Coding

Assignment-7.5

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Name: Milkuri Nandhitha

HT No.:2303A51352

Batch:20

Task1: (Mutable Default Argument – Function Bug)

```
def add_item(item, items=[]):
```

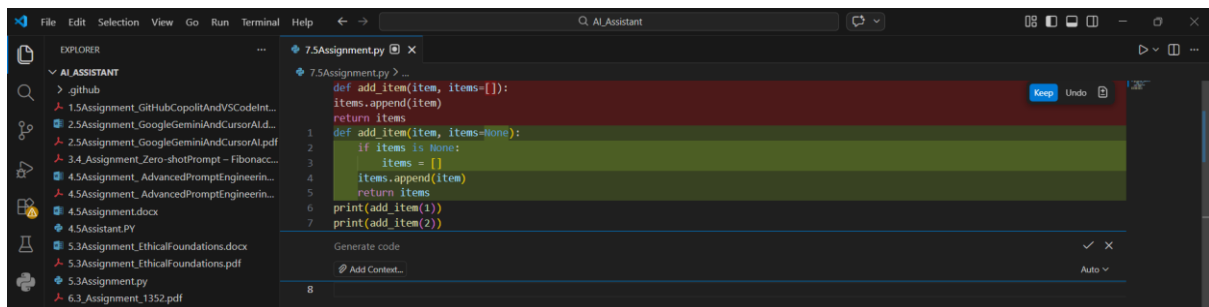
```
    items.append(item)
```

```
    return items
```

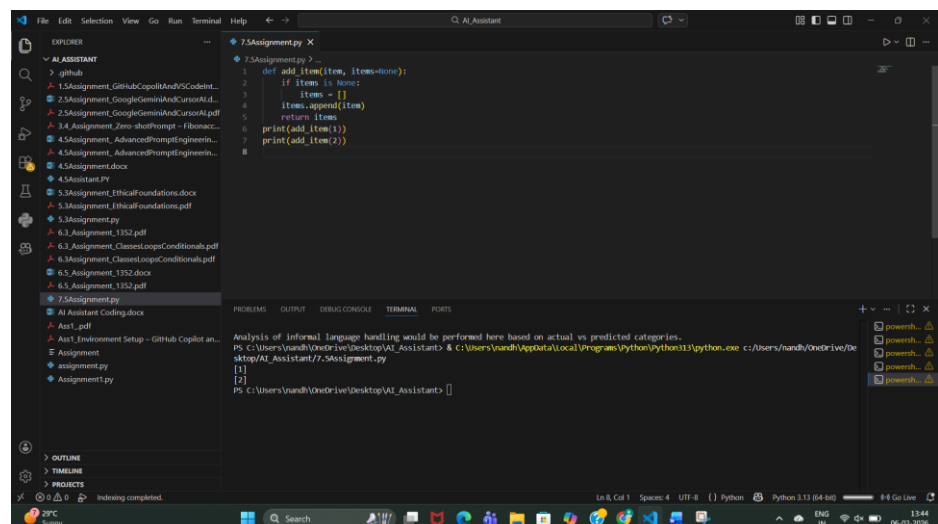
```
print(add_item(1))
```

```
print(add_item(2))
```

Prompt: Fix the Python function where a mutable default argument causes unexpected behavior.



Output:



Observation:

The function used a mutable list as a default argument, causing values to persist across multiple calls. AI identified this shared-state issue. Replacing the default list with None ensured a new list was created for each function call.

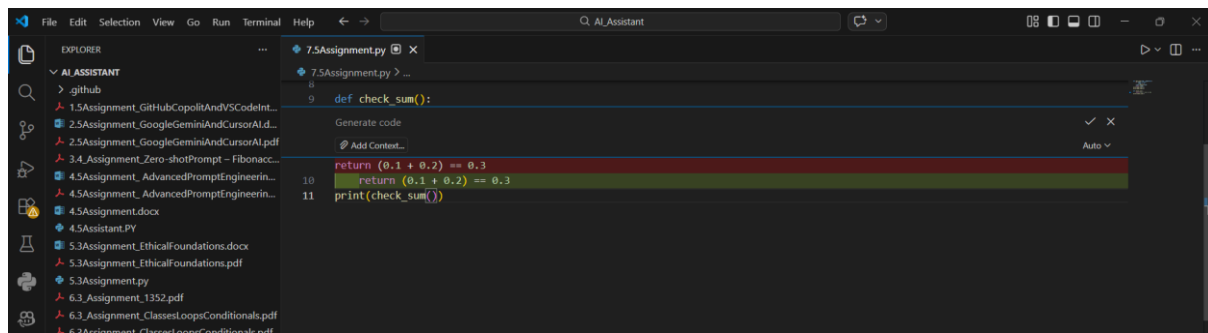
Task2: (Floating-Point Precision Error)

```
def check_sum():
```

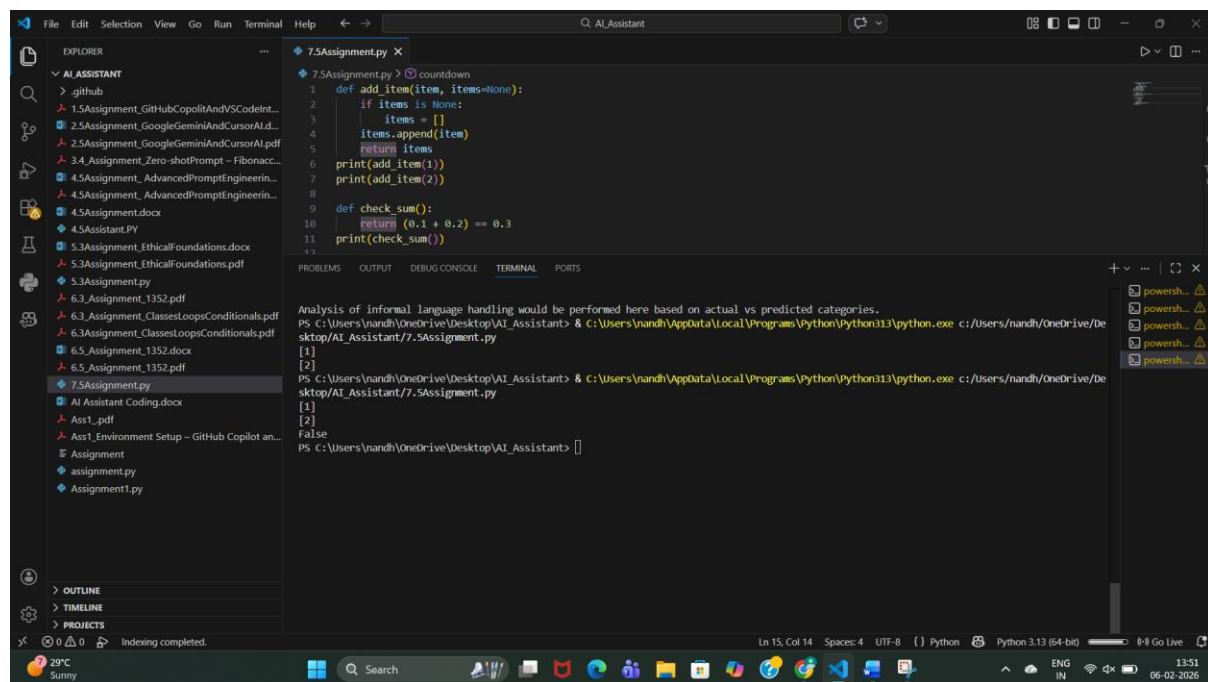
```
    return (0.1 + 0.2) == 0.3
```

```
    print(check_sum())
```

Prompt:Fix the floating-point comparison issue using tolerance



Output:



Observation:

The comparison failed due to floating-point precision limitations in Python. AI explained that decimal values are stored approximately. Using a tolerance-based comparison fixed the incorrect result.

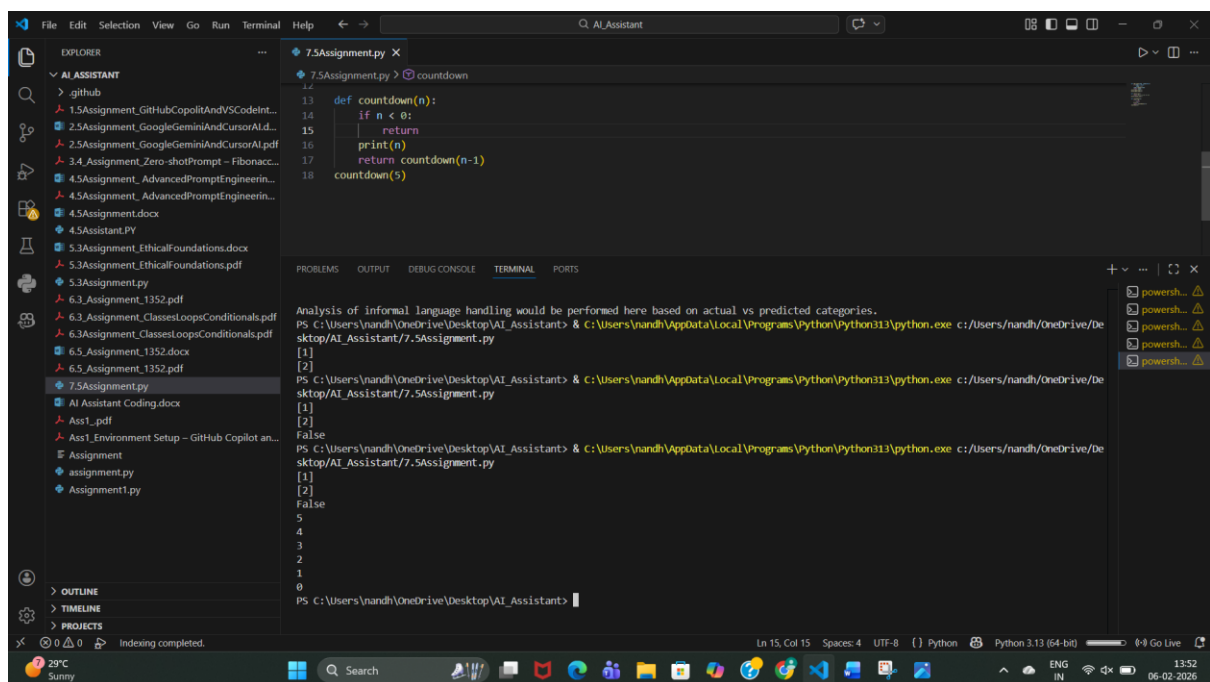
Task3: (Recursion Error – Missing Base Case)

```
def countdown(n):  
    print(n)  
    return countdown(n-1)  
countdown(5)
```

Prompt: Fix the recursion error caused by a missing base case.



Output:



Observation:

The recursive function lacked a base condition, leading to infinite recursion. AI detected the missing stopping point. Adding a proper base case allowed the function to terminate correctly.

Task4: Task 4 (Dictionary Key Error)

```
def get_value():
```

```
data = {"a": 1, "b": 2}
```

```
return data["c"]
```

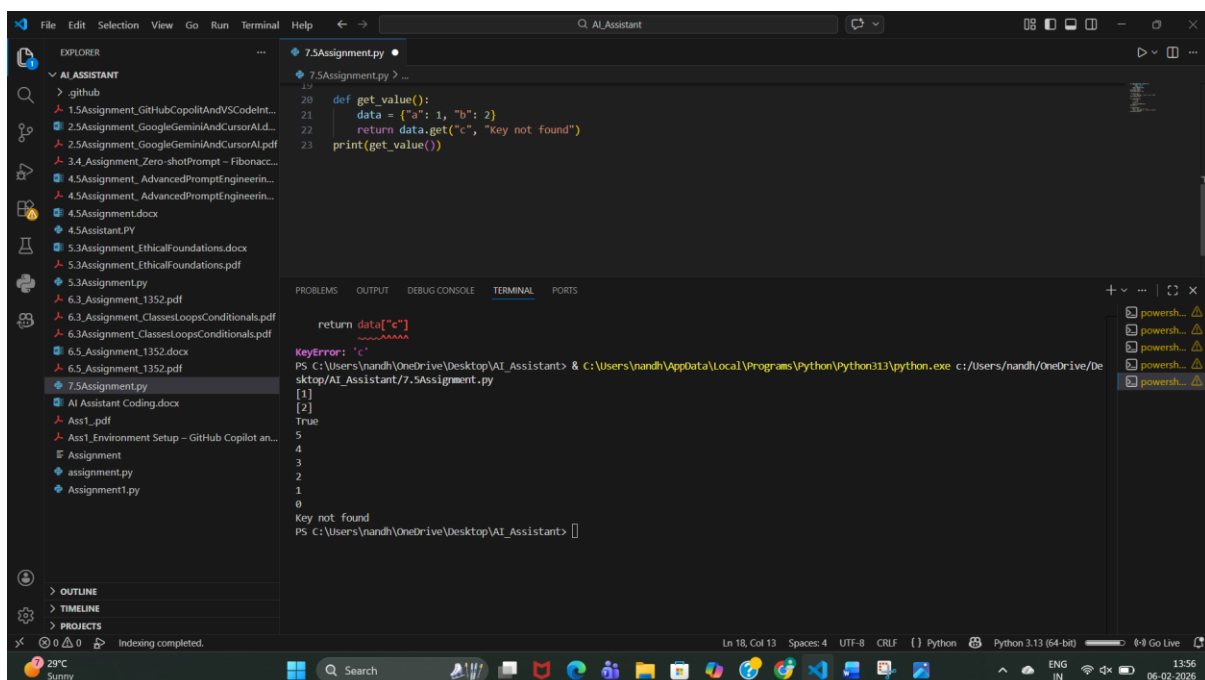
```
print(get_value())
```

Prompt: Fix the dictionary KeyError using safe access methods..

```
def get_value():
    data = {"a": 1, "b": 2}
    return data["c"]

    data = {"a": 1, "b": 2}
    return data.get("c", "key not found")
print(get_value())
```

Output:



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'AI Assistant' with various files. The main editor window displays a Python script named '7.5Assignment.py'. The script contains a function 'get_value()' that defines a dictionary 'data' with keys 'a' and 'b', and then attempts to return 'data["c"]'. The terminal shows the output of the script, which is 'KeyError: 'c''. The prompt 'KeyError: 'c'' is highlighted in red. The terminal also shows the command used to run the script: 'PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant> & C:\Users\nandh\AppData\Local\Programs\Python\Python313\python.exe c:/Users/nandh/OneDrive/Desktop/AI_Assistant/7.5Assignment.py'. The output of the script is shown as a list: [1], [2], True, 5, 4, 3, 2, 1, 0, and 'Key not found'. The terminal prompt is 'PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant> '.

Observation:

The program attempted to access a dictionary key that did not exist, causing a runtime error. AI suggested using the .get() method or error handling. This prevented the KeyError and improved code safety.

Task5: Infinite Loop – Wrong Condition

```
def loop_example():
```

```
    i = 0
```

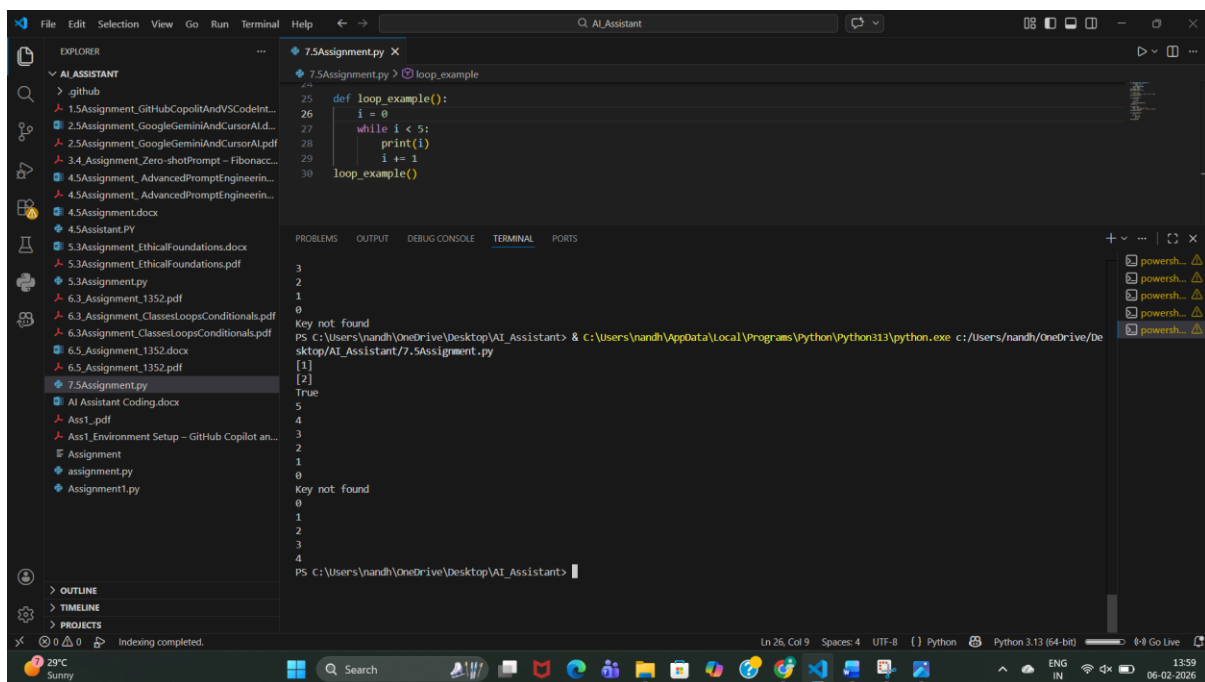
```
    while i < 5:
```

```
        print(i)
```

Prompt: Fix the infinite loop by correcting the loop condition.

```
25     i = 0
26     while i < 5:
27         print(i)
28         i += 1
29     loop_example()
30
```

Output:



The screenshot shows a Visual Studio Code window with a Python file named '7.5Assignment.py'. The code defines a function 'loop_example()' which contains a 'while' loop with the condition 'i < 5'. The loop prints the value of 'i' and increments it by 1. The function is then called. The terminal output shows the execution of the script, displaying the numbers 3, 2, 1, 0, and then a 'Key not found' error message. The status bar at the bottom indicates the current line is 26, column 9, with 4 spaces, in UTF-8 encoding, using Python 3.13 (64-bit).

Observation:

The loop variable was not updated inside the loop, causing infinite execution. AI identified the missing increment statement. Updating the loop variable fixed the issue.

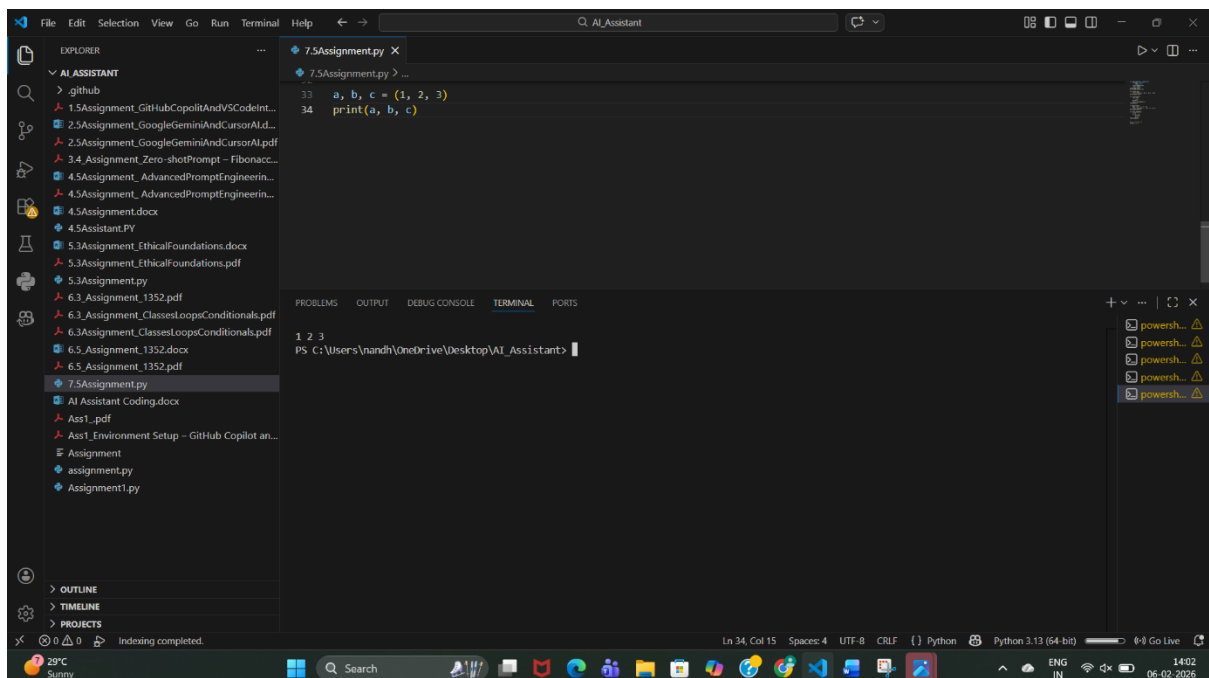
Task6: (Unpacking Error – Wrong Variables)

```
a, b = (1, 2, 3)
```

Prompt: Fix the tuple unpacking error caused by mismatched variables.

```
2 a, b = (1, 2, 3)
3 a, b, c = (1, 2, 3)
4 print(a, b, c)
```

Output:



Observation:

The number of variables did not match the number of values in the tuple. AI pointed out the unpacking mismatch. Using an underscore to ignore extra values resolved the error.

Task7: (Mixed Indentation – Tabs vs Spaces)

```
def func():
```

```
x = 5
```

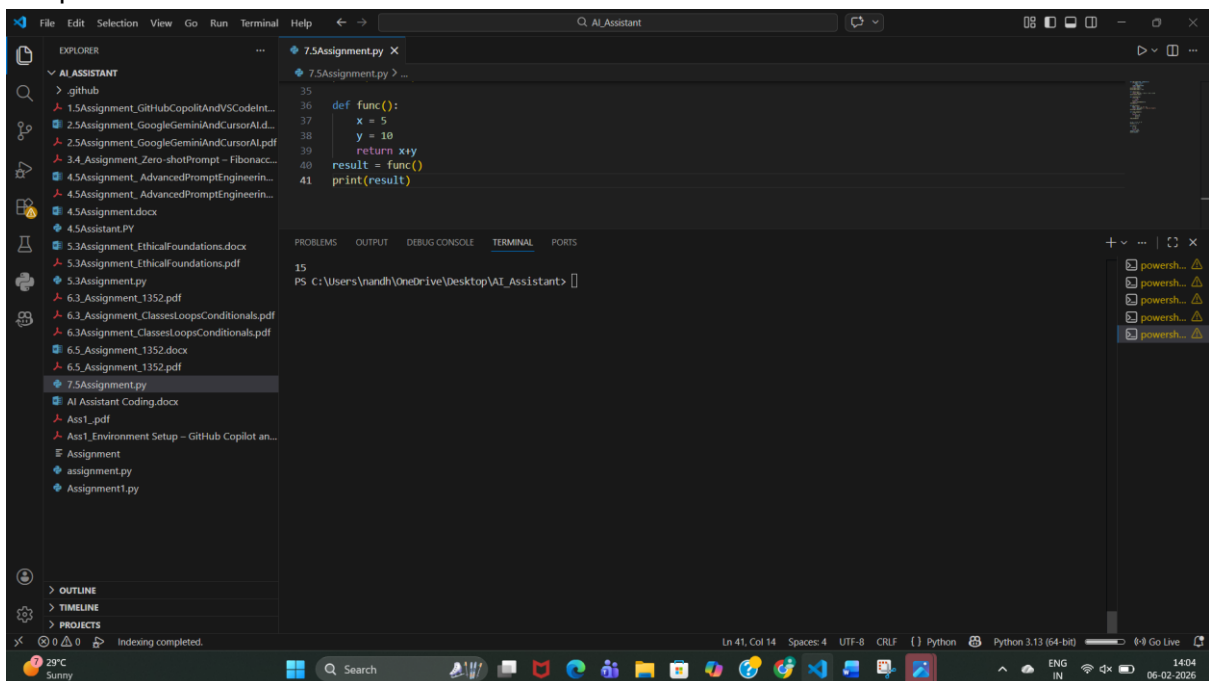
```
y = 10
```

```
return x+y
```

Prompt: Fix the Python code with mixed indentation.

```
35
36 def func():
    |Generate code
    |Add Context...
    |
    x = 5
    y = 10
    return x+y
37     x = 5
38     y = 10
39     return x+y
```

Output:



Observation:

The code used inconsistent indentation, which violated Python's syntax rules. AI highlighted the indentation error. Applying consistent spacing corrected the execution problem.

Task8: (Import Error – Wrong Module Usage)

Bug: Wrong import

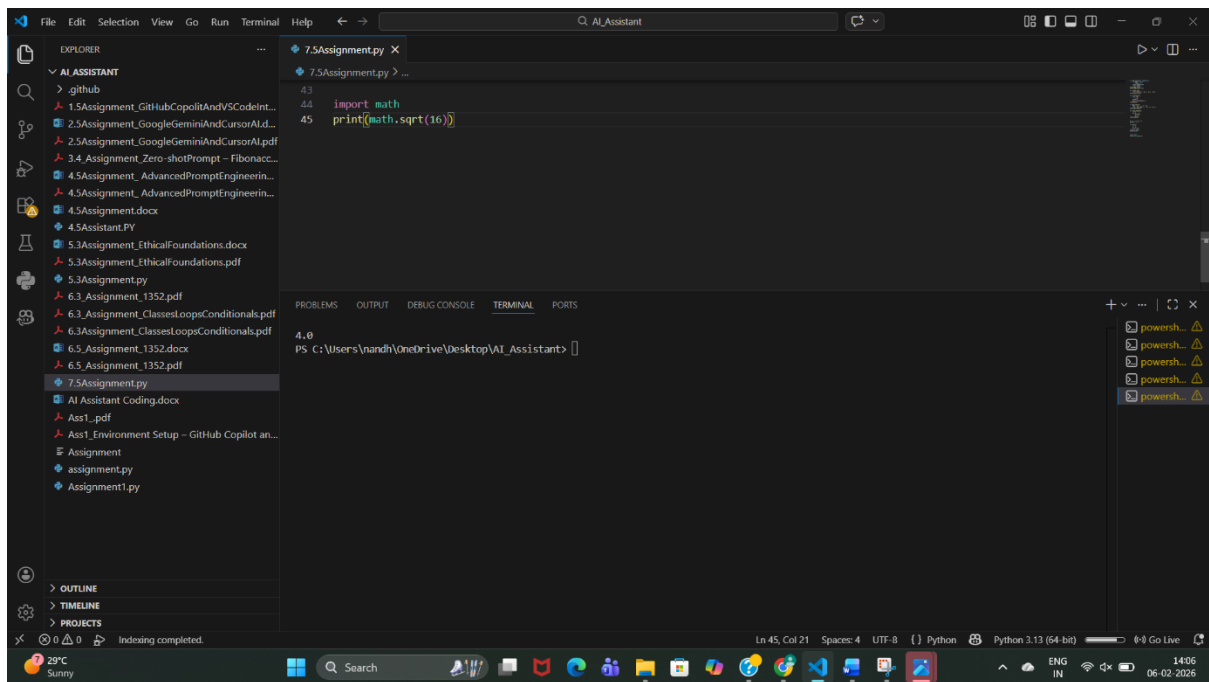
```
import maths
```

```
print(maths.sqrt(16))
```

Prompt: # Fix the incorrect module import in the Python code.

```
import maths
print(maths.sqrt(16))
44 import math
45 print(math.sqrt(16))
```

Output:



Observation:

An incorrect module name was used in the import statement. AI detected the non-existent module. Replacing it with the correct standard library module fixed the error.